

**САНКТ-ПЕТЕРБУРГСКИЙ НАЦИОНАЛЬНЫЙ
ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО**

Дисциплина: Бэк-энд разработка

Отчет

Лабораторная работа

Выполнил:

Андронов Денис

Группа

К3342

Проверил:

Добряков Д. И.

Санкт-Петербург

2026 г.

Задача

ЛР4: Развёртывание приложения на удалённом сервере

Срок: 20.05.2026

Задание:

- провести предварительную настройку удалённого сервера (установка необходимых зависимостей);
- провести развёртывание приложения на удалённом сервере;
- настроить nginx в режиме обратного прокси для обеспечения доступа к вашему приложению извне.

Ход работы

Компонент	Роль
setup-server.sh	однократная подготовка ОС: пакеты, Docker, nginx, firewall
deploy.sh	повторяемый запуск приложения через Docker Compose
install-nginx.sh + nginx/restorator.conf	публикация HTTP снаружи, проксирование на контейнер
docker-compose.prod.yml	переопределение портов для production: наружу только gateway на localhost

Все bash-скрипты начинаются с `#!/usr/bin/env bash` и используют `set -euo pipefail`: при любой ошибке выполнение прерывается, неиспользуемые переменные и сбои в пайпах не игнорируются.

3. Скрипт `deploy/setup-server.sh`

Назначение: первичная настройка сервера под Ubuntu 22.04/24.04.

Логика по шагам:

1. Проверка прав root - сравнение EUID с нулём; без sudo скрипт завершается с подсказкой.

2. `DEBIAN_FRONTEND=noninteractive` - установка пакетов без интерактивных вопросов.
3. `apt-get update` - обновление индекса репозитория.
4. Базовые пакеты: `ca-certificates`, `curl`, `git`, `nginx`, `ufw` - для HTTPS при скачивании Docker, клонирования репозитория, веб-сервера и файрвола.
5. Установка Docker по официальной схеме для Ubuntu:
 - каталог `/etc/apt/keyrings`;
 - ключ с `download.docker.com`;
 - запись `deb ...` с `VERSION_CODENAME` из `/etc/os-release`;
 - пакеты `docker-ce`, `docker-cli`, `containerd.io`, `docker-buildx-plugin`, `docker-compose-plugin` (Compose v2 как подкоманда `docker compose`).
6. `systemctl enable --now` для `docker` и `nginx` - автозапуск и немедленный старт.
7. UFW: разрешены SSH (OpenSSH), HTTP/HTTPS (Nginx Full), включение правил с `--force`.
8. Каталог `/opt/restorator` - зарезервировано место под приложение; владелец - `SUDO_USER`, если скрипт вызван через `sudo`.

Скрипт не клонирует репозиторий сам - это сделано осознанно: путь к проекту и ветка могут отличаться.

4. Скрипт `deploy/deploy.sh`

Назначение: сборка образов и запуск стека из каталога `restorator-microservices`.

Логика:

1. `SCRIPT_DIR` - каталог, где лежит сам скрипт (`deploy/`), через `dirname` и `pwd`.

2. PROJECT_ROOT - родитель deploy/, то есть корень lab4.
3. COMPOSE_DIR - restorator-microservices, где находятся docker-compose.yml и docker-compose.prod.yml.
4. COMPOSE=(docker compose -f ... -f ...) - массив команд: единый список файлов для всех вызовов, чтобы не путать override и базовый compose.
5. down --remove-orphans - остановка и удаление контейнеров прошлого запуска, снятие «осиротевших» контейнеров после смены compose.
6. up -d --build - фоновый режим и пересборка при изменении Dockerfile/контекста.
7. Цикл ожидания - до 30 итераций с паузой 5 с; проверка curl -sf http://127.0.0.1:4000/health (тихий режим, только HTTP 2xx). Успех - вывод docker compose ps и выход с кодом 0.
8. При таймауте - последние 50 строк логов api-gateway и выход с кодом 1.

Так скрипт одновременно деплоит приложение и даёт простую диагностику при сбое.

5. Файл restorator-microservices/docker-compose.prod.yml

Назначение: production-надстройка к основному docker-compose.yml.

Содержание:

- Для RabbitMQ и PostgreSQL в секции ports указано !override [] - полностью убираются пробросы портов из базового файла; БД и брокер доступны только внутри Docker-сети.
- Для api-gateway указано ports: !override с одной строкой 127.0.0.1:4000:4000.

Зачем !override: при объединении двух compose-файлов без override Docker мог бы добавить второй проброс порта 4000 к уже объявленному в базовом docker-compose.yml (4000:4000 на всех интерфейсах), что приводило к ошибке «address already in use». Ключ !override заменяет

список портов целиком: наружу с хоста попадает только один биндинг на loopback, что согласовано с nginx на том же хосте.

6. Шаблон `deploy/nginx/restorator.conf`

Назначение: виртуальный хост nginx на порту 80 (IPv4 и IPv6).

Параметры:

- `server_name` `SERVER_NAME` - заглушка; при установке подставляется IP или домен.
- `client_max_body_size` 10m - ограничение тела запроса для API.
- `location /` с `proxy_pass http://127.0.0.1:4000` - весь внешний HTTP уходит в API Gateway в Docker.
- Заголовки `Host`, `X-Real-IP`, `X-Forwarded-For`, `X-Forwarded-Proto` - корректная передача клиента и схемы приложению.
- `Upgrade / Connection` - на случай WebSocket.

7. Скрипт `deploy/install-nginx.sh`

Назначение: установка сайта nginx из шаблона и перезагрузка конфигурации.

Логика:

1. Проверка `root` и наличия аргумента (IP или домен).
2. Путь к шаблону относительно скрипта: `deploy/nginx/restorator.conf`.
3. `sed` - подстановка `SERVER_NAME` значением первого аргумента в файл `/etc/nginx/sites-available/restorator`.
4. Симлинк в `sites-enabled`, удаление дефолтного сайта `default`, чтобы не конфликтовал порт 80.
5. `nginx -t` - проверка синтаксиса; при ошибке скрипт не выполнит `reload`.
6. `systemctl reload nginx` - применение без полного рестарта.

Вывод

Скрипты разбиты по ответственности: подготовка ОС, запуск контейнеров, настройка reverse проху. Используются строгий режим bash, официальный репозиторий Docker, Compose с двумя файлами и явный !override портов для безопасности и устранения конфликта портов. Отчёт можно дополнить скриншотами выполнения команд и выводом docker compose ps после успешного деплоя.